

① How many threads would you use for processing a million files?

→ as many as CPU cores (usually)

→ if file processing involves I/O, use more threads than cores (because many threads will be blocked waiting for disk I/O, while others can run)

⇒ avoids excessive context switching.

CPU-bound → #threads $\approx$ #cores

I/O-bound → can use more threads than cores.

② see 12 jun 2026 (917)

③ Why ^{do} I/O operations cause a process to move from RUNNING state → WAIT state?

→ The process has to pause itself while it sends/gets data from I/O.

→ During this time it cannot continue execution, so it releases CPU and moves from RUN to WAIT until the I/O operation completes. (it leaves the processor for other processes)

ASC! ④ How is the address calculation done in the absolute fixed partition alloc?

ABSOLUTE FIXED PARTITION ALLOCATION: each partition has a fixed starting address known in advance.

↗ RELOCATABLE

physical address = partition start address + logical address

ex: partition start address: 1000

logical address: 50 (accessed by process)

2) phys. addr = 1000 + 50 = 1050

offset inside the partition.

The program is loaded into a predetermined partition and uses absolute memory addresses directly, so no address translation is required.

MEMORY ALLOCATION

I. FIXED partition allocation

— RAM is divided beforehand —

ABSOLUTE
FIXED

RELOCATABLE
FIXED

III. PAGING (no external fragm.)

linear addr = base + offset

↓ paging
physical address

II. DYNAMIC partition allocation

— Partitions created as needed —

⇒ external fragmentation?

First fit

Best fit

Worst fit

Next fit

IV. SEGMENTATION (external fragm.)

linear addr = base + offset

V. SEGMENTATION + PAGING (nowadays)

⑤ ④ and ③ of the First-fit placement policy vs Worst-fit.

First-fit: ④ faster because it stops at the first block large enough for the request. (searches less memory)

③ may increase external fragmentation because of the many small unusable holes near the beginning of the memory. (wasted free space)

⑥ The most prioritary memory page that the NRU replacement policy chooses as victim page?

NRU = not recently used.

=> classifies pages using: (R, H)

↓
referenced bit modified bit

Classes: Class 0: (0, 0) - not referenced, not modif. page

Class 1: (0, 1) - not ref., modif. page

Class 2: (1, 0) - ref., not modif. page

Class 3: (1, 1) - ref., modif. page.

=> NRU chooses a victim from the lowest-numbered non-empty class.

↓
class 0.

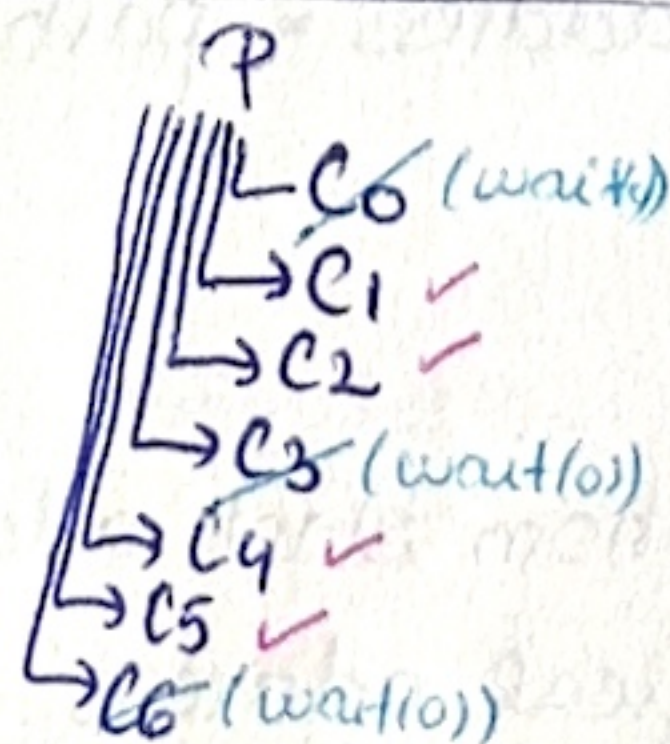
⑦ size of block = B ; size of address = A ;

How many blocks are addressed by the triple indirect addressing of an i-mode?

A block can store: $\frac{B}{A}$ addresses.

triple => $\left(\frac{B}{A}\right)^3$
indirect

(*)



Right before calling the last wait(0), there were 5 active processes => MAX.

⑧ REGEX that accepts lines that contain "a" but don't contain "b".

grep -E '^[^b]*a[^b]*\$' filename
start line 0 or more characters ≠ b must contain an "a"

⑨ for(i=0; i<7; i++) {

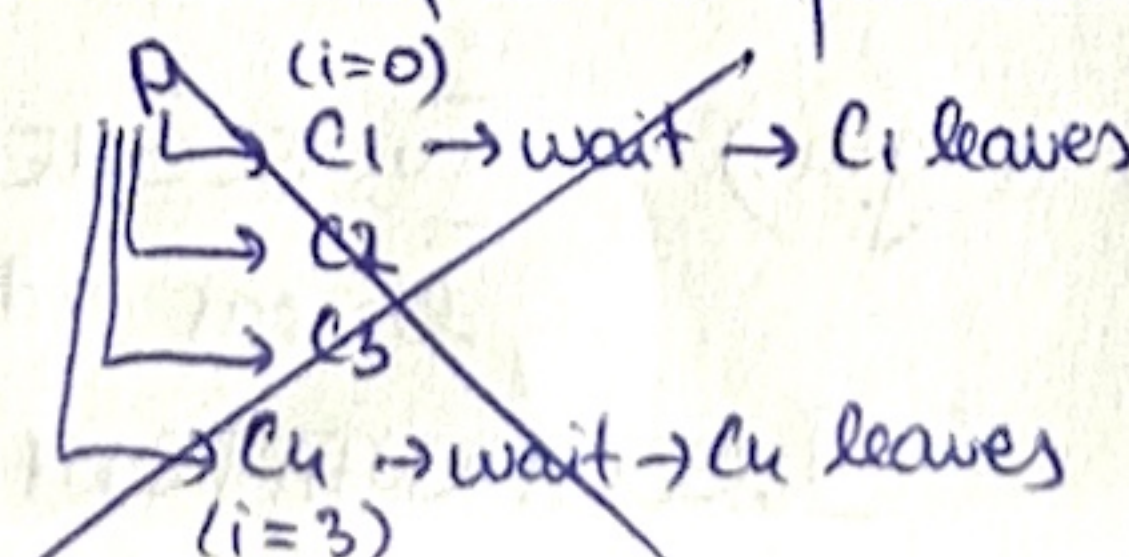
if (fork() == 0) {

sleep(rand()%10);

exit(0);

if (i%3 == 0) { wait(0); }

→ Max. nr. of child processes? (that can coexist)



(*)